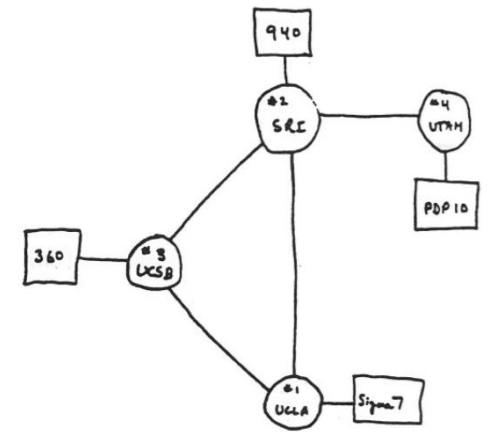
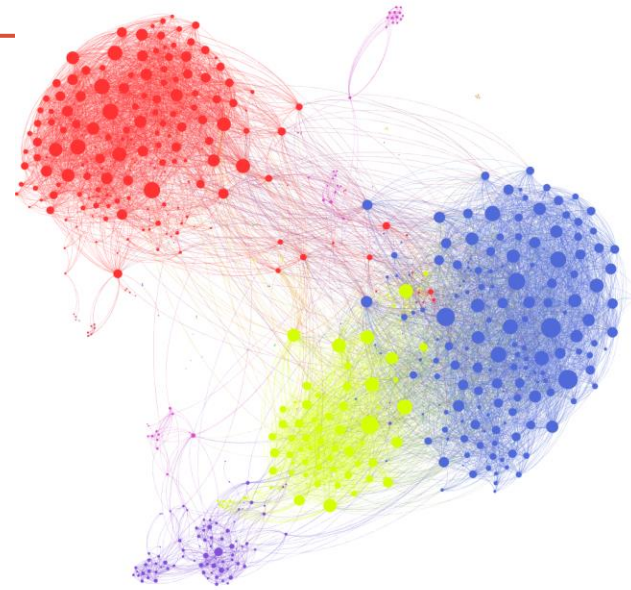
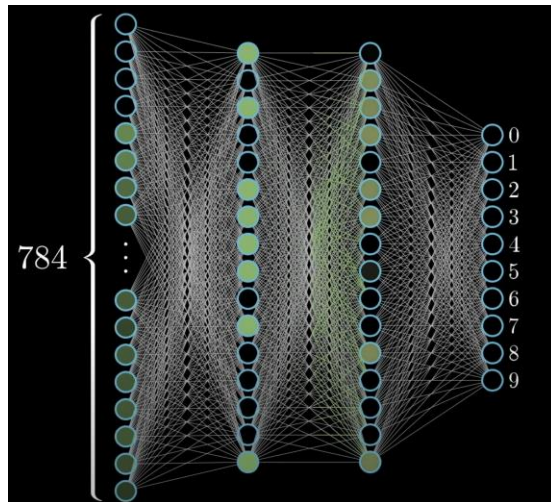


GRAPH REPRESENTATION

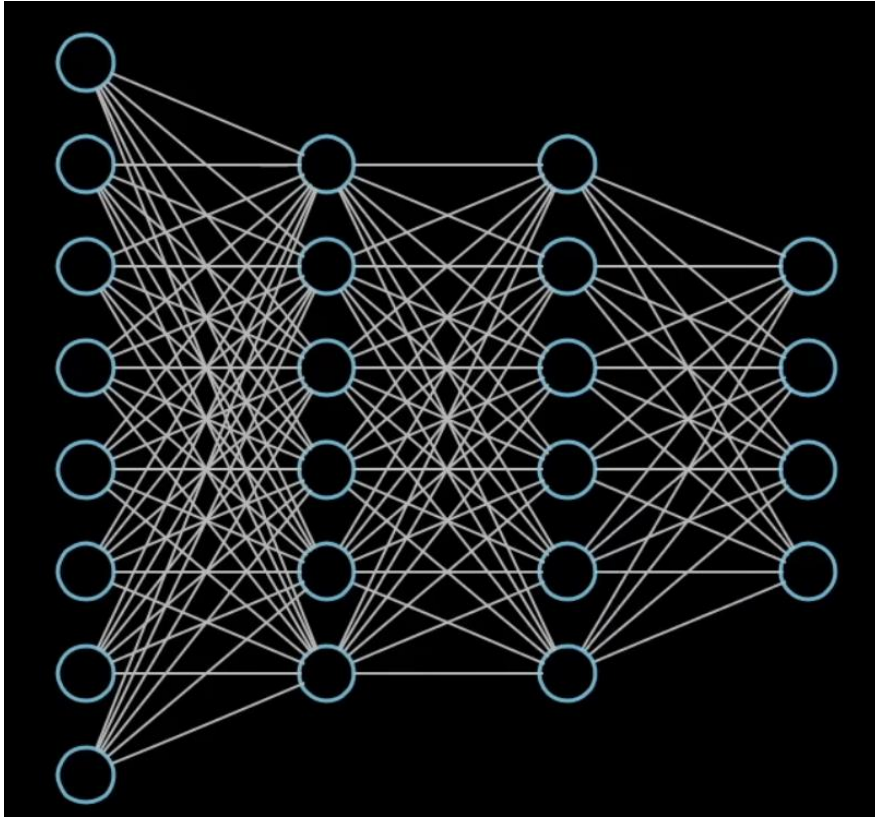


THE ARPA NETWORK

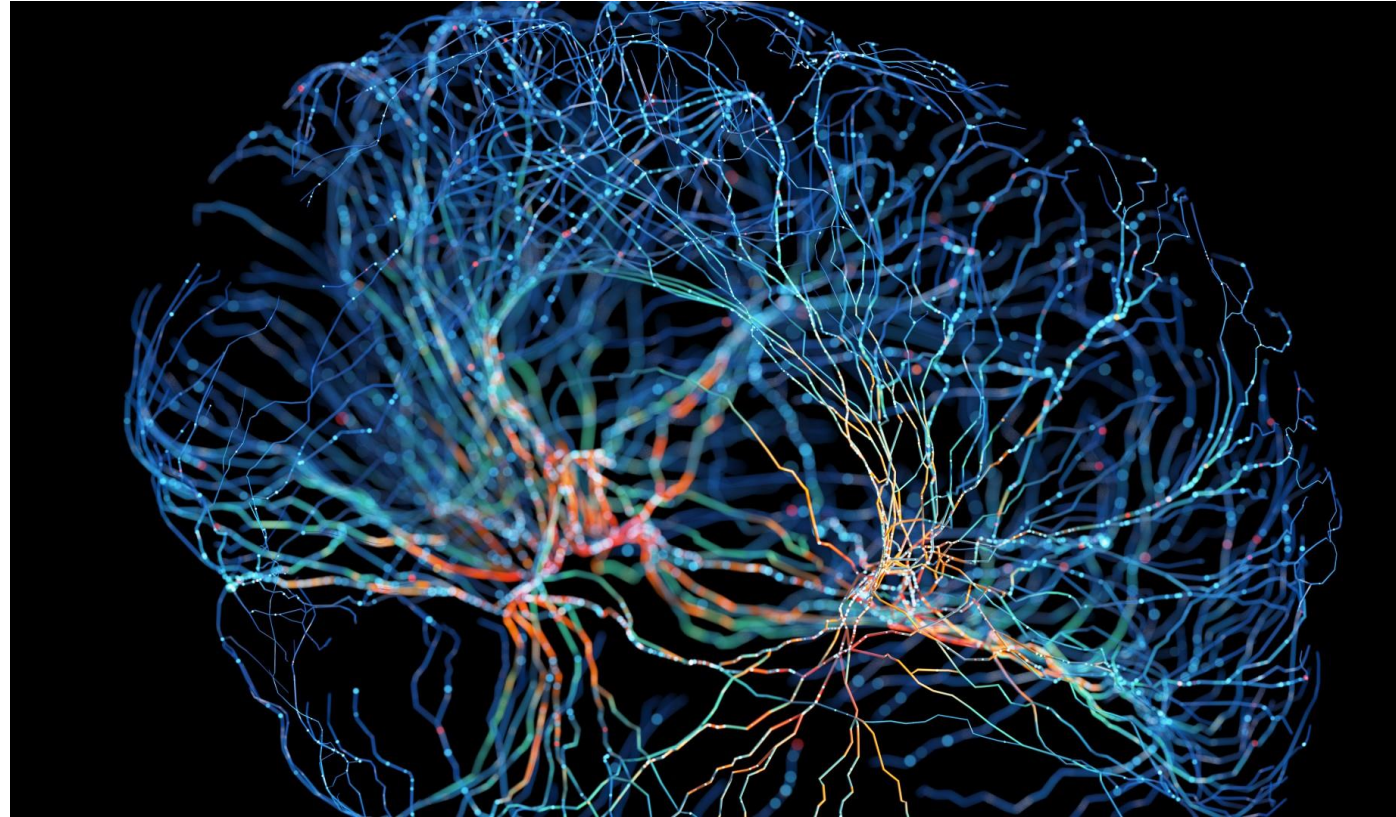
DEC 1969

4 NODES

Neural Networks: Biologically inspired structure



Neural Network: Collection of connected neurons modeled after the brain

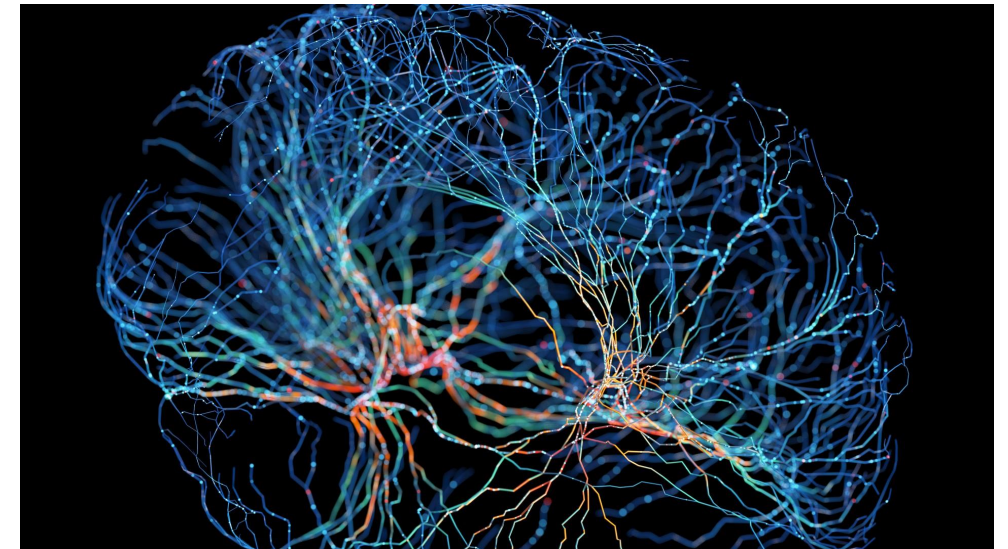
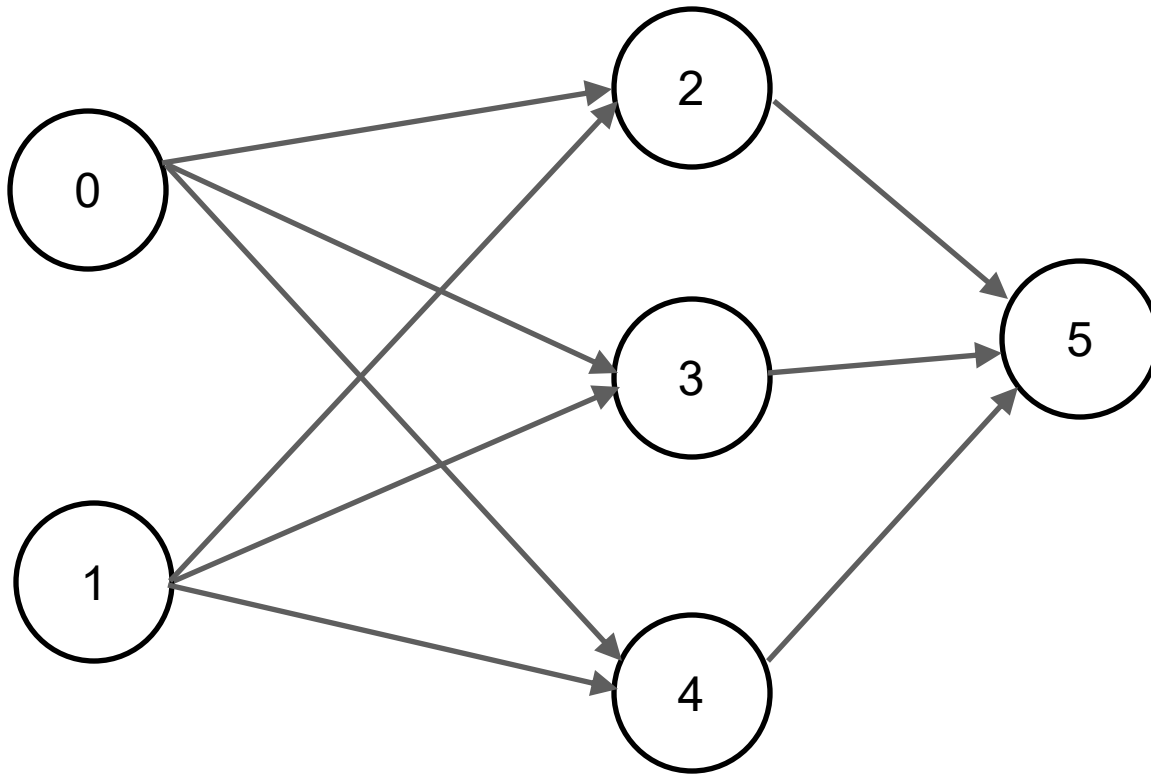


Human brain: 86 billion neurons
Each neuron is connected to 1000 other neurons

Video intro to neural networks: <https://youtu.be/aircAruvnKk?feature=shared>

Neural Network: Set of neurons and connections

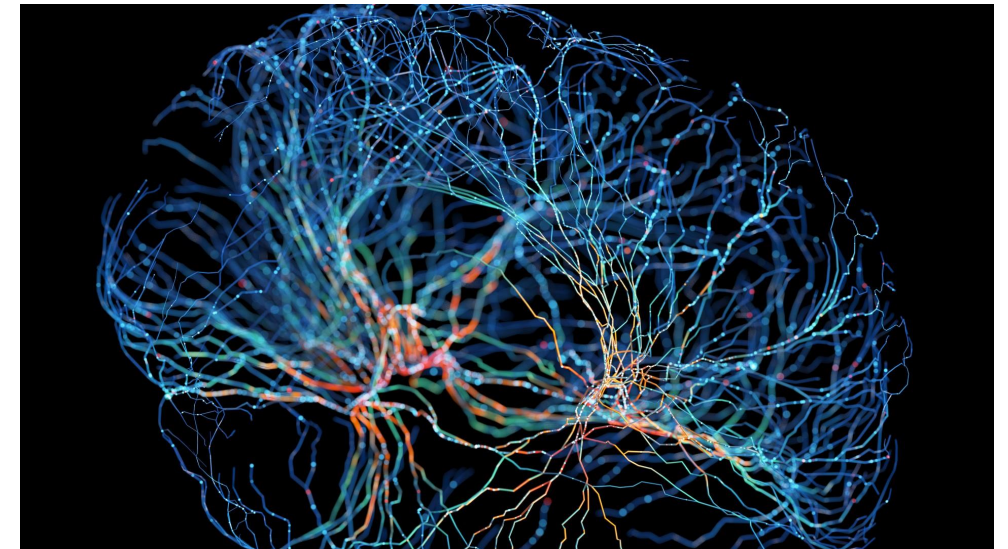
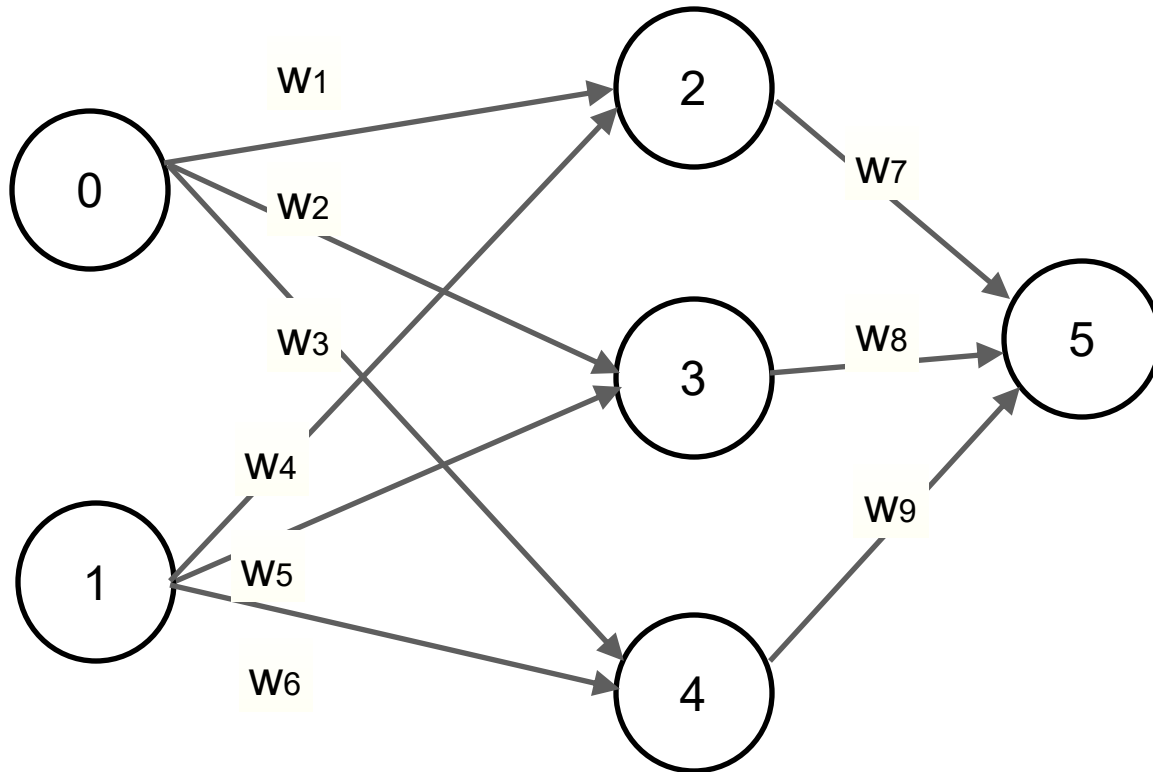
- Connections are directed (one way)
- Brain synaptic connections are one-way but feedback loops can exist



Neural Network: Collection of connected neurons modeled after the brain

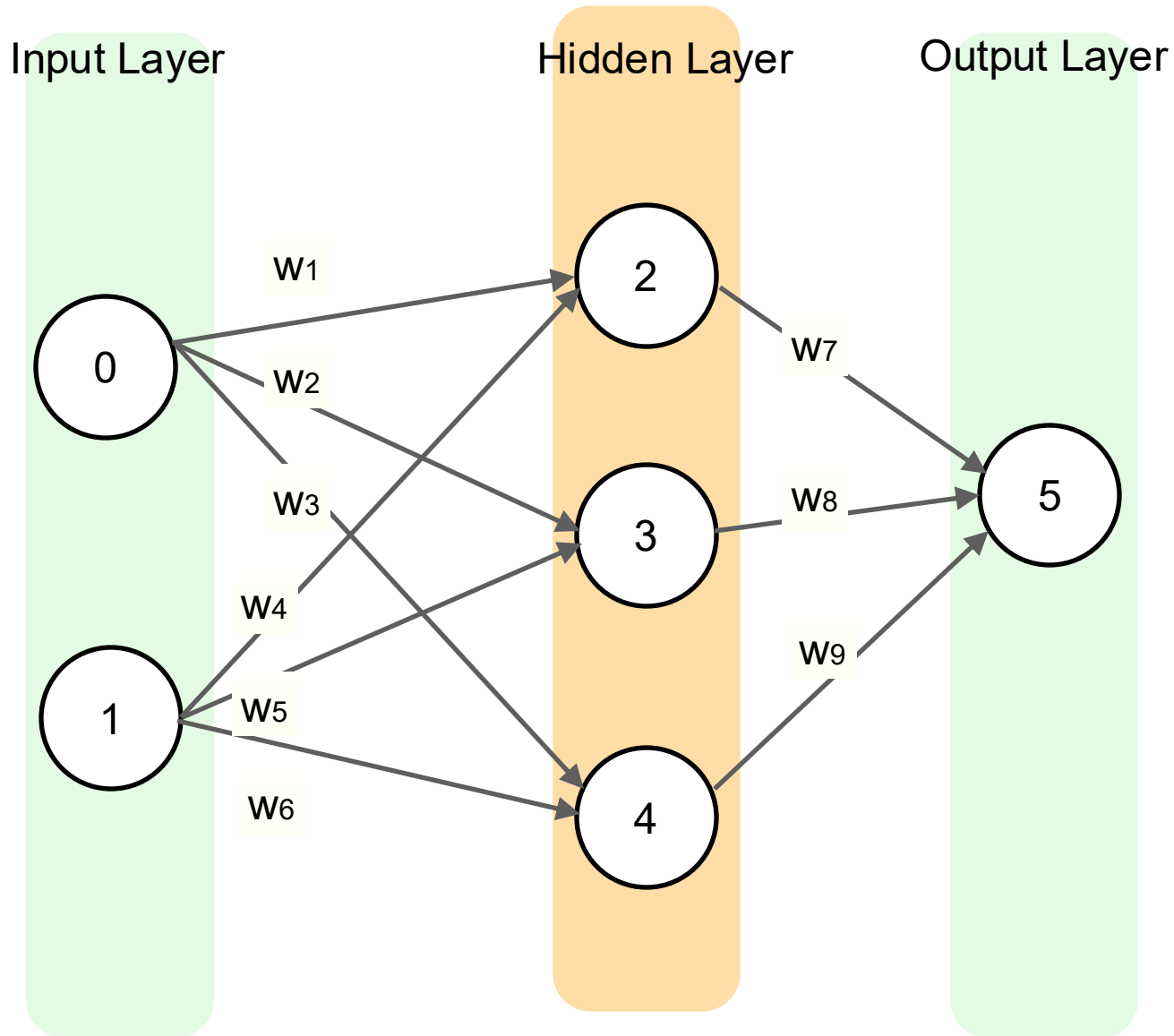
Neural Network: Connections have weights

- Connections have weights
- Cells that fire together wire together



Neural Network: Collection of connected neurons modeled after the brain

Feed-forward neural net (PA03)



A **feedforward neural network (FNN)** is the simplest type of neural network where data flows in one direction — from input to output — without looping back.

NN Terminology

Neurons

Connection

Input Layer

Hidden Layer

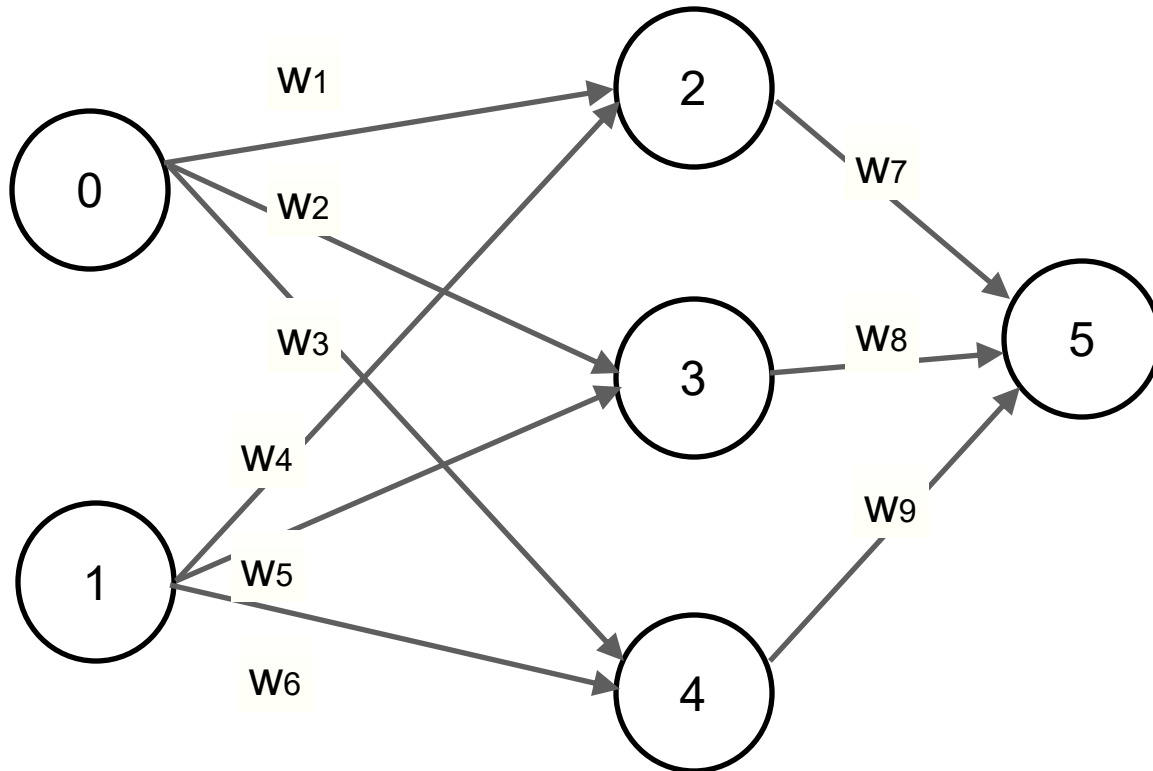
Output Layer

Neuron Info
value, bias, activation

How would you represent the neural net using the data structures learned so far?

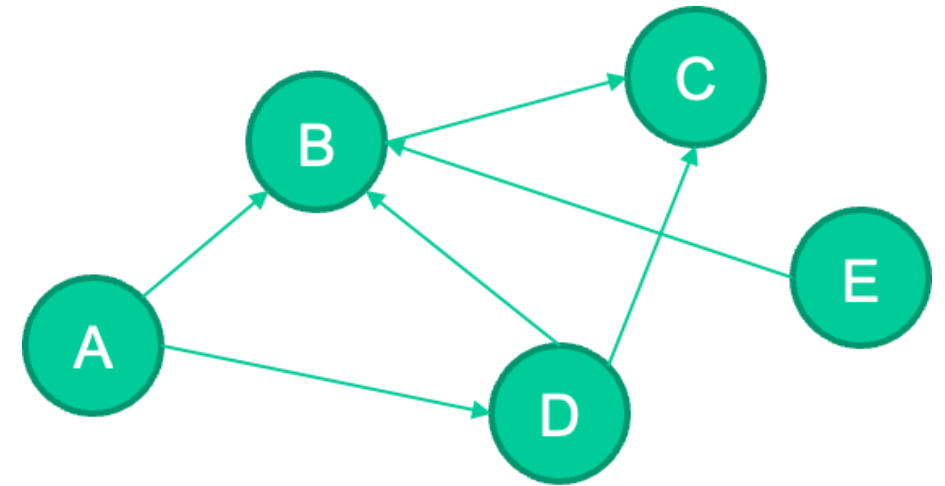
Neural Networks as Graphs

(PA03) Model a neural network as a _____ graph.



NN is a set of **neurons and connections**

- Connections are directed (one-way)
- Connections have weights (strength of connection)



Graph is a set of nodes (vertices) and edges

- Directed graph: Edges go one way
- Undirected graph: Edges go both ways
- Weighted graph: Edges have weights

Graph Terminology and Notation

Graph $G = \{V, E\}$

- Vertices $V = \{0, 1, 2, 3, \dots, n - 1\}; n = |V|$
- Edges $E = \{(u, v) \mid u \in V, v \in V\}; m = |E|$

List the set of vertices and edges for the example graph

$V =$

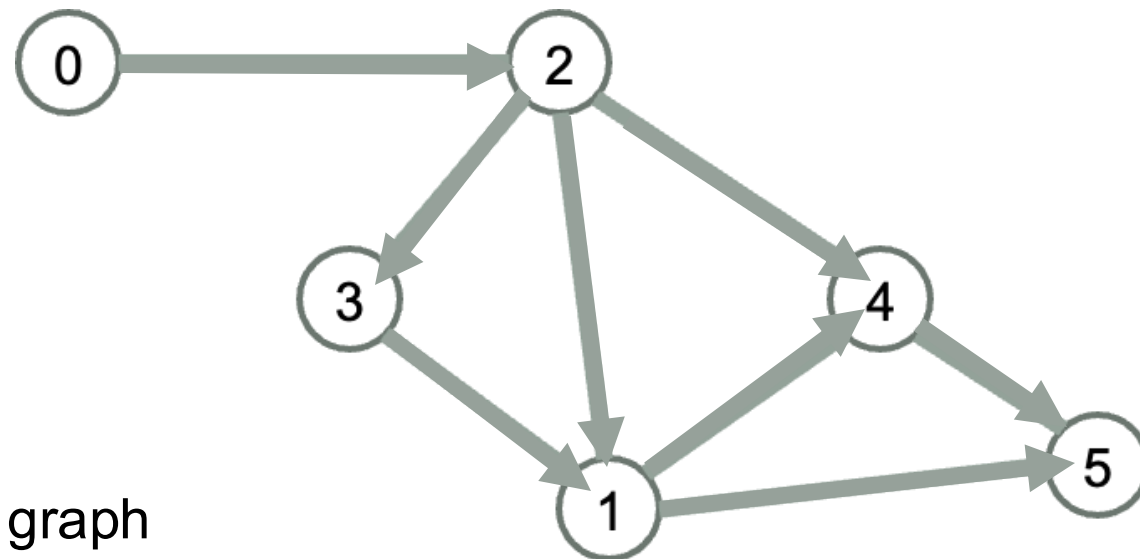
$E =$

$n = |V| =$

$m = |E| =$

Discuss (5 mins):

How would you represent the set of vertices and edges using the data structures learned so far?



Representing vertices

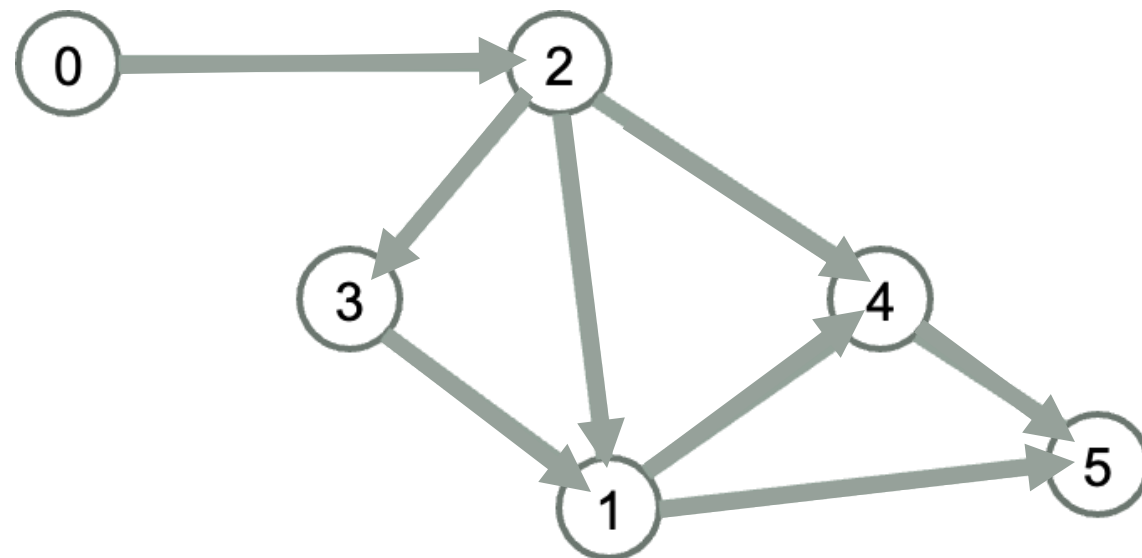
Graph $G = \{V, E\}$

$$V = \{0, 1, 2, 3, \dots, n - 1\}; n = |V|$$

Your graph has vertices labeled 0 through $n - 1$.

What's the best way to store the set of vertices?

- A. Vector, why?
- B. Hashtable, why?
- C. BST, why?
- D. Something else?



Consider:

1. What do you need to do with the vertices?

(Look them up? Loop through them?)

2. Which is faster and uses less memory—vector or hashtable?

3. Does it help that the vertex labels are just 0, 1, 2, ..., $n - 1$?

Convince your neighbor why your pick (vector? Hashtable? BST? something else?) is the best.

Representing vertices with associated info

```
struct NodeInfo {  
    double value;  
    double bias;  
    // other fields associated with a neuron  
};
```

```
std::vector<NodeInfo> nodes(n); // where n is known ahead of time
```

OR

```
std::vector<NodeInfo*> nodes(n);
```

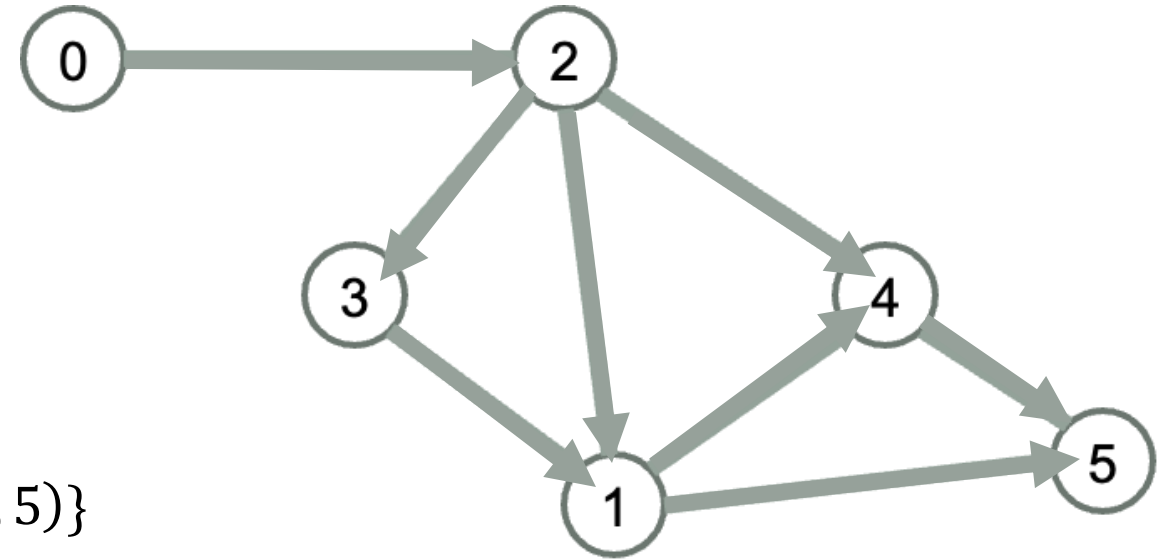
Representing edges using an adjacency list

An **adjacency list** is a way to represent a graph where each vertex stores a list of its neighboring vertices (the ones it's connected to by an edge).

$$G = (V, E)$$

$$V = \{0, 1, 2, 3, 4, 5\}$$

$$E = \{(0,2), (2,3), (2,1), (2,4), (3,1), (1,4), (1,5), (4,5)\}$$



Complete the adjacency list representation of the example graph.

Given a vertex (v), how efficiently can you find all its neighbors?

Define neighbors of a vertex (v).

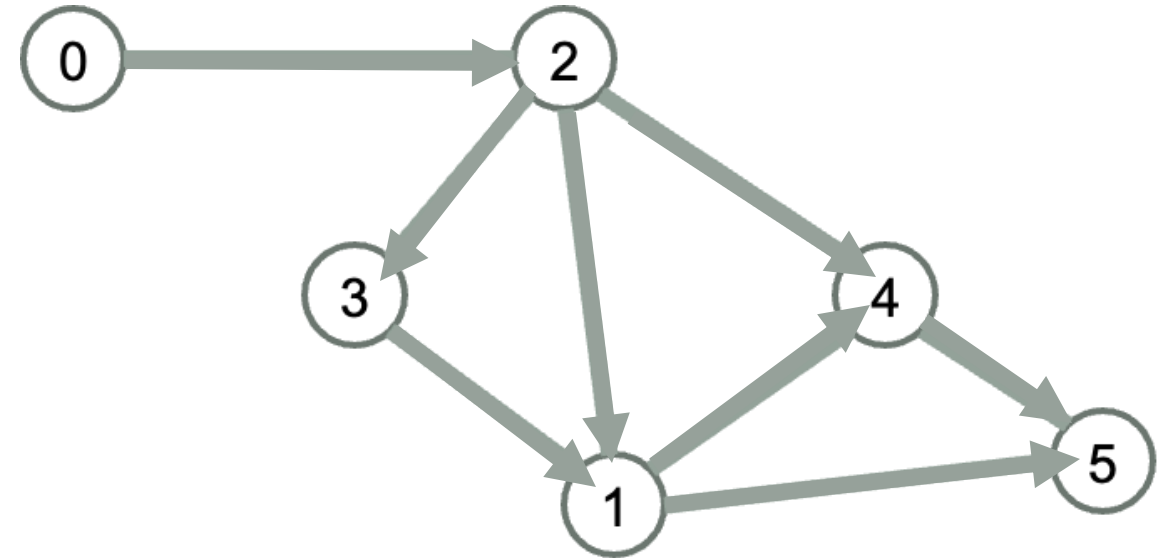
Does this look like any other data structure you have seen so far?

Representing edges using an adjacency list

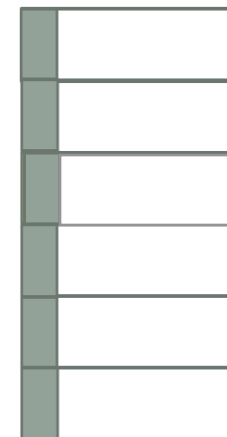
```
class graph{  
    ...  
private:  
    vector<NodeInfo> nodes;  
    _____ adjlist;  
};
```

Choose the ADT to represent adjlist

- A. `vector<int>`
- B. `vector<unordered_set<int>>`
- C. `list<vector<int>>`
- D. `vector<list<int>>`
- E. `set<list<int>>`



adjlist



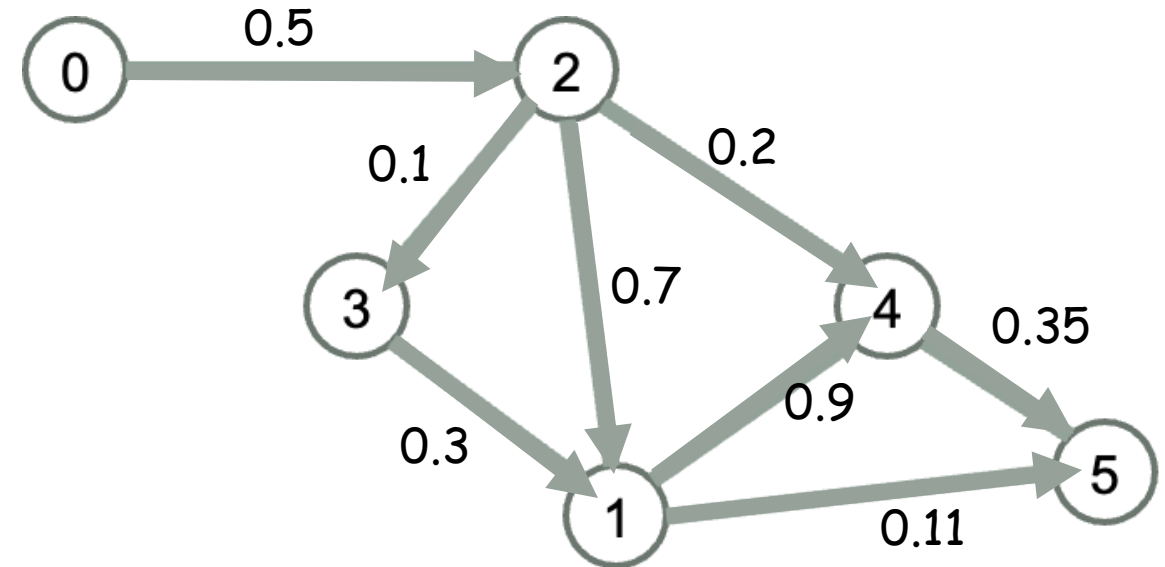
We need an efficient way to find all the neighbors of a given vertex.

What if edges had weights?

```
class graph{
  ...
private:
  vector<NodeInfo> nodes;
  _____ adjlist;
};
```

How would you represent adjlist for a weighted graph?

```
// Use when edges have no weights
vector<unordered_set<int>> adjlist;
```



adjlist

Understanding the Graph and NeuralNetwork classes

```
typedef std::vector<std::unordered_map<int, Connection> > AdjList;
```

```
class Graph {

    public:
        Graph();
        Graph(int size);
        // Constructors and destructor

        // TODO: graph methods
        void updateNode(int id, NodeInfo n);
        NodeInfo* getNode(int id) const;
        void updateConnection(int v, int u, double w);

    protected:
        // protected to give NeuralNetwork access

        // adjacency list containing weights for edges.
        AdjList adjacencyList;

        // vector storing node info
        std::vector<NodeInfo*> nodes;

        //Other functions

};
```

```
class NeuralNetwork : public Graph {

    public:

        // Constructors and public functions

    private:

        // each index of layers holds a vector which contains the id's of every
        node in that layer.
        std::vector<std::vector<int> > layers;

        // contains ids of input nodes
        std::vector<int> inputNodeIds;

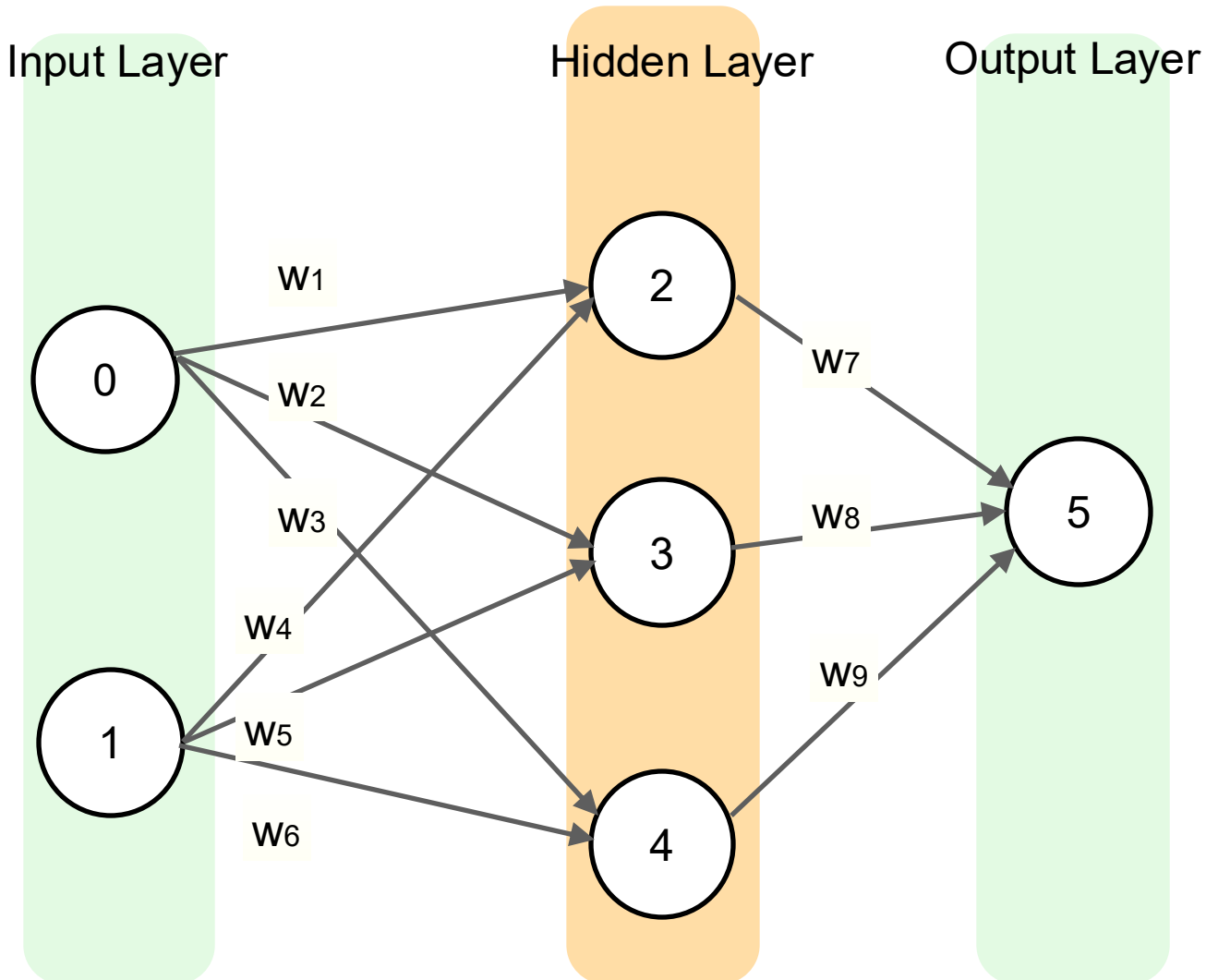
        // contains ids of output nodes
        std::vector<int> outputNodeIds;

        // since NeuralNetwork inherits from Graph, you can imagine all of the
        graph members here as well...

};
```

Neural Network structure for upcoming assignment

```
typedef std::vector<std::unordered_map<int, Connection> > AdjList;
```



```
class Graph {  
    // Representation related info  
    AdjList adjacencyList;  
    std::vector<NodeInfo*> nodes;  
};
```

```
void test_algorithm() {  
    cout << "test_algorithm" << endl;  
    NeuralNetwork nn(6);  
  
    NodeInfo n0("ReLU", 0, -0.2);  
    NodeInfo n1("ReLU", 0, 0.2);  
    NodeInfo n2("identity", 0, 0);  
    NodeInfo n3("sigmoid", 0, 0.98);  
    NodeInfo n4("ReLU", 0, 0.11);  
    NodeInfo n5("identity", 0, 0);  
  
    nn.updateNode(0, n0);  
    nn.updateNode(1, n1);  
    nn.updateNode(2, n2);  
    nn.updateNode(3, n3);  
    nn.updateNode(4, n4);  
    nn.updateNode(5, n5);  
  
    nn.updateConnection(2, 1, 0.1);  
    nn.updateConnection(2, 4, 0.2);  
    nn.updateConnection(2, 0, 0.3);  
    nn.updateConnection(5, 1, 0.4);  
    nn.updateConnection(5, 4, 0.5);  
    nn.updateConnection(5, 0, 0.6);  
    nn.updateConnection(1, 3, 0.7);  
    nn.updateConnection(4, 3, 0.8);  
    nn.updateConnection(0, 3, 0.9);  
  
    nn.setInputNodeIds({2, 5});  
    nn.setOutputNodeIds({3});  
}
```

Activity: Draw the final neural net and its representation in memory